

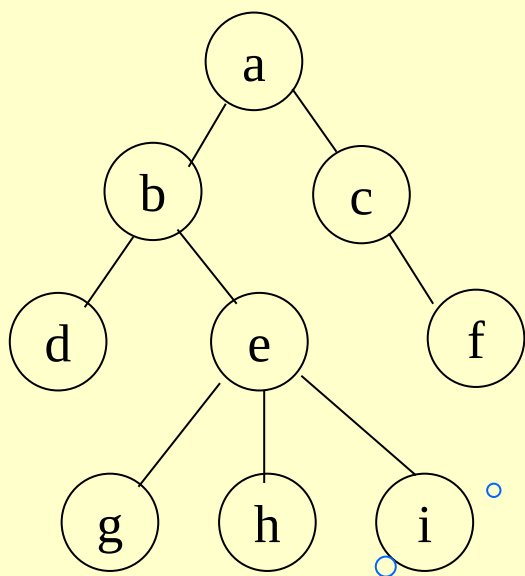
6.4 树和森林

6.4.1 树的存储结构

- 双亲表示法

- 实现：定义结构数组存放树的结点，每个结点含两个域：

- 数据域：存放结点本身信息
- 双亲域：指示本结点的双亲结点在数组中位置



如何找孩子结点

■特点：找双亲容易，找要扫描整个表。

	data	parent
0	a	-1
1	b	0
2	c	0
3	d	1
4	e	1
5	f	2
6	g	4
7	h	4
8	i	4
9		

• 孩子表示法

- 1、多重链表：每个结点有多个指针域，分别指向其子树的根
结点同构：结点的指针个数相等，为树的度D

data	child1	child2		childD
------	--------	--------	-------	--------

结点不同构：结点指针个数不等，为该结点的度d

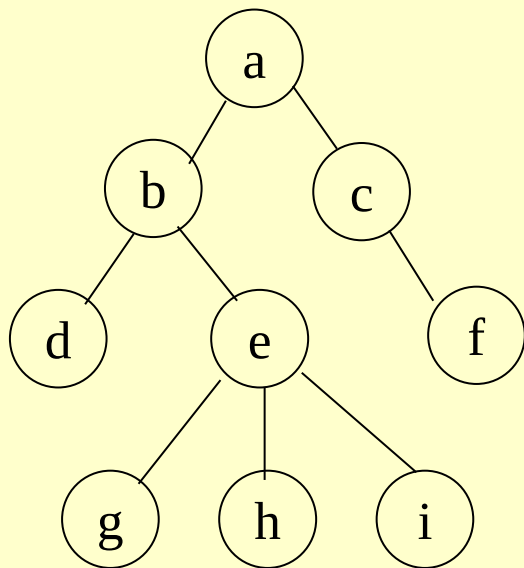
data	degree	child1	child2		childd
------	--------	--------	--------	-------	--------

- 2、孩子链表：每个结点的孩子结点用单链表存储，再用含n个元素的结构数组指向每个孩子链表

```
孩子结点: typedef struct ctnode
{   int child; //该结点在表头数组中下标
    struct ctnode *next; //指向下一孩子结点
}*childptr;
```

```
表头结点: typedef struct
{ telemtype data; //数据域
  childptr firstchild; //孩子链表头指针
}ctbox;
```





```

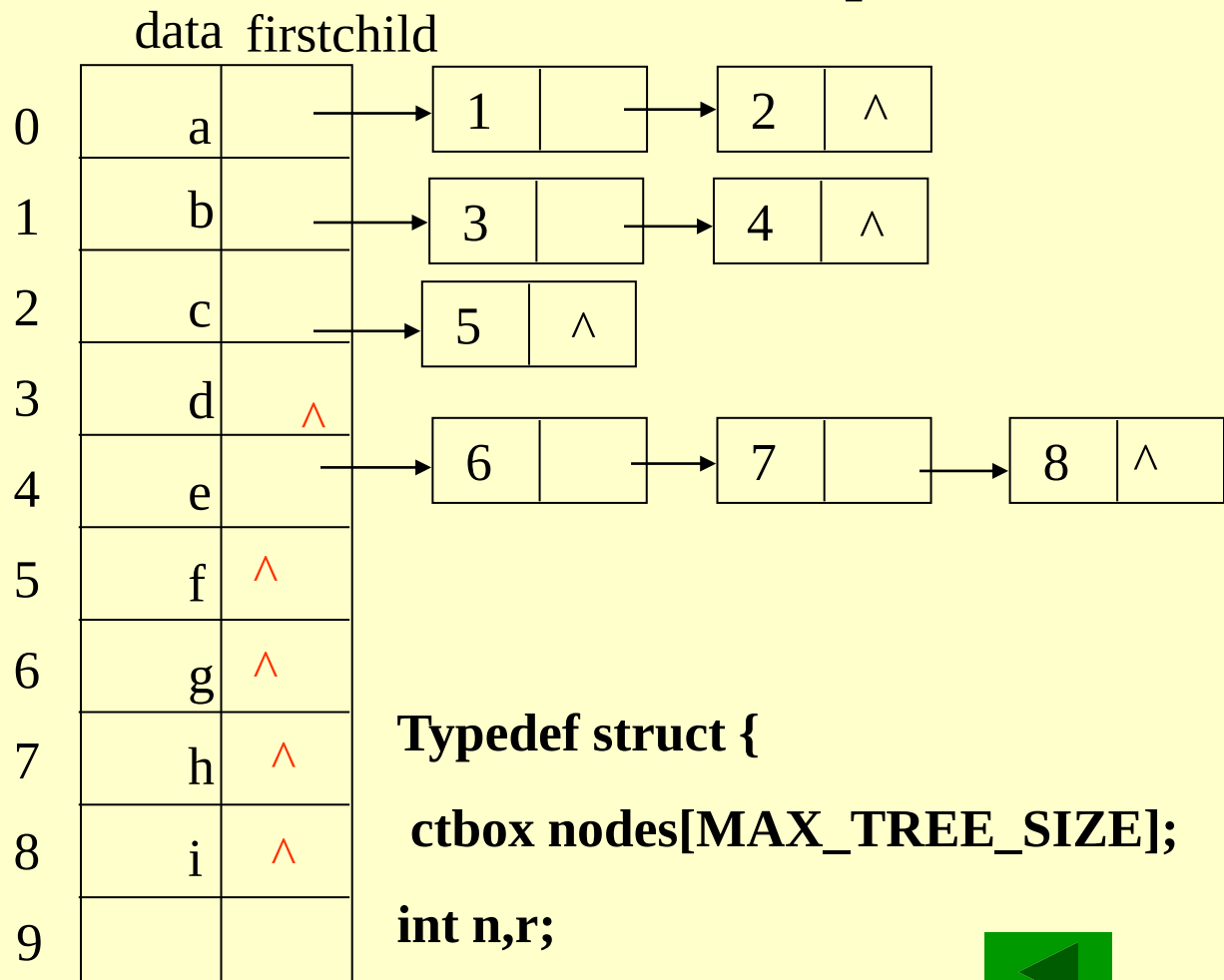
typedef struct ctnode
{ int child;
  struct ctnode *next;
  }*childptr;

```

```

typedef struct
{ telemtype data;
  childptr firstchild;
  }ctbox;

```



```

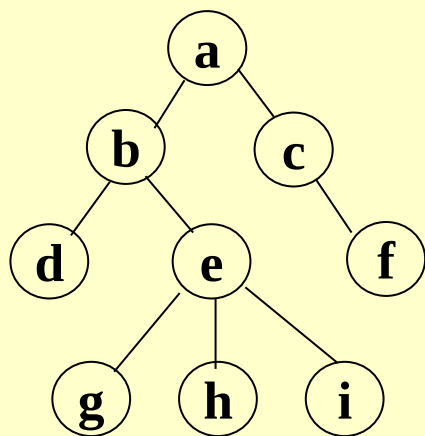
Typedef struct {
  ctbox nodes[MAX_TREE_SIZE];
  int n,r;
  }ctree

```

如何找双亲结点



□ 带双亲的孩子链表



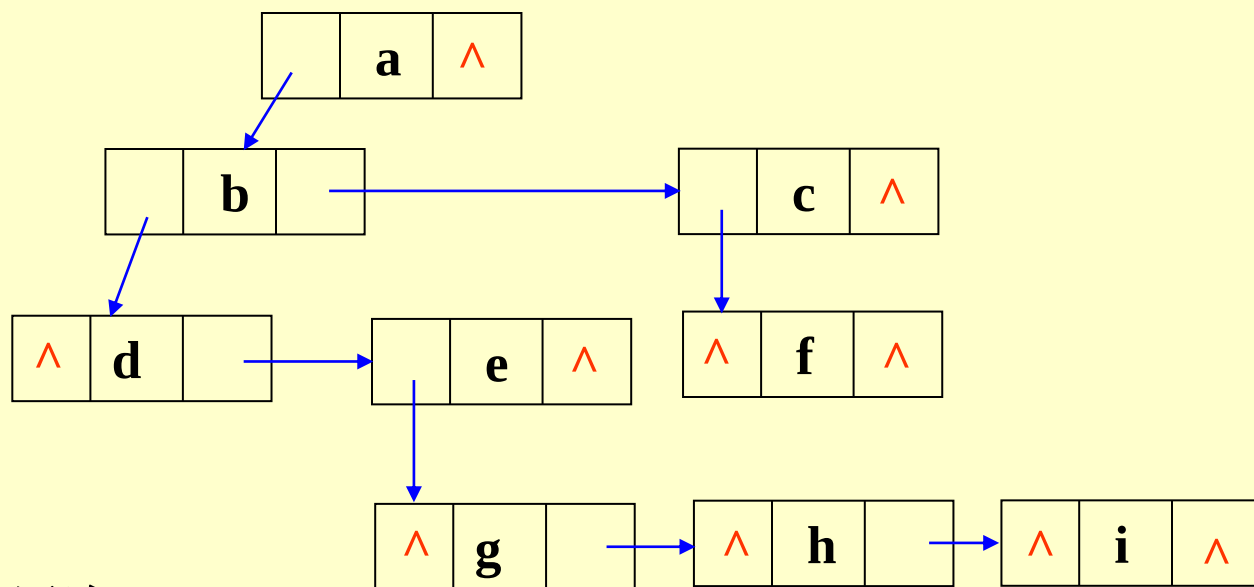
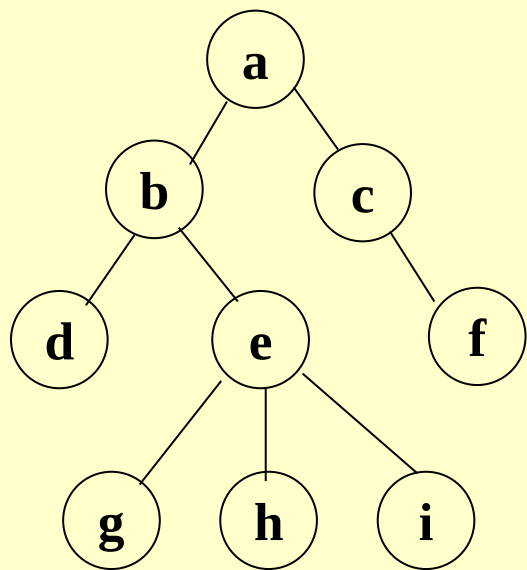
	data	parent	firstchild
0	a	-1	
1	b	0	
2	c	0	
3	d	1	^
4	e	1	
5	f	2	^
6	g	4	^
7	h	4	^
8	i	4	^

1		→	2	^
3		→	4	^
5	^			
6		→	7	
			8	^

• 3、孩子兄弟表示法（二叉树表示法）

- 实现：用二叉链表作树的存储结构，链表中每个结点的两个指针域分别指向其第一个孩子结点和下一个兄弟结点

```
typedef struct csnode  
{ elemtype data;  
    struct csnode *firstchild,  
    *nextsibling;  
}csnode,*cstree;
```



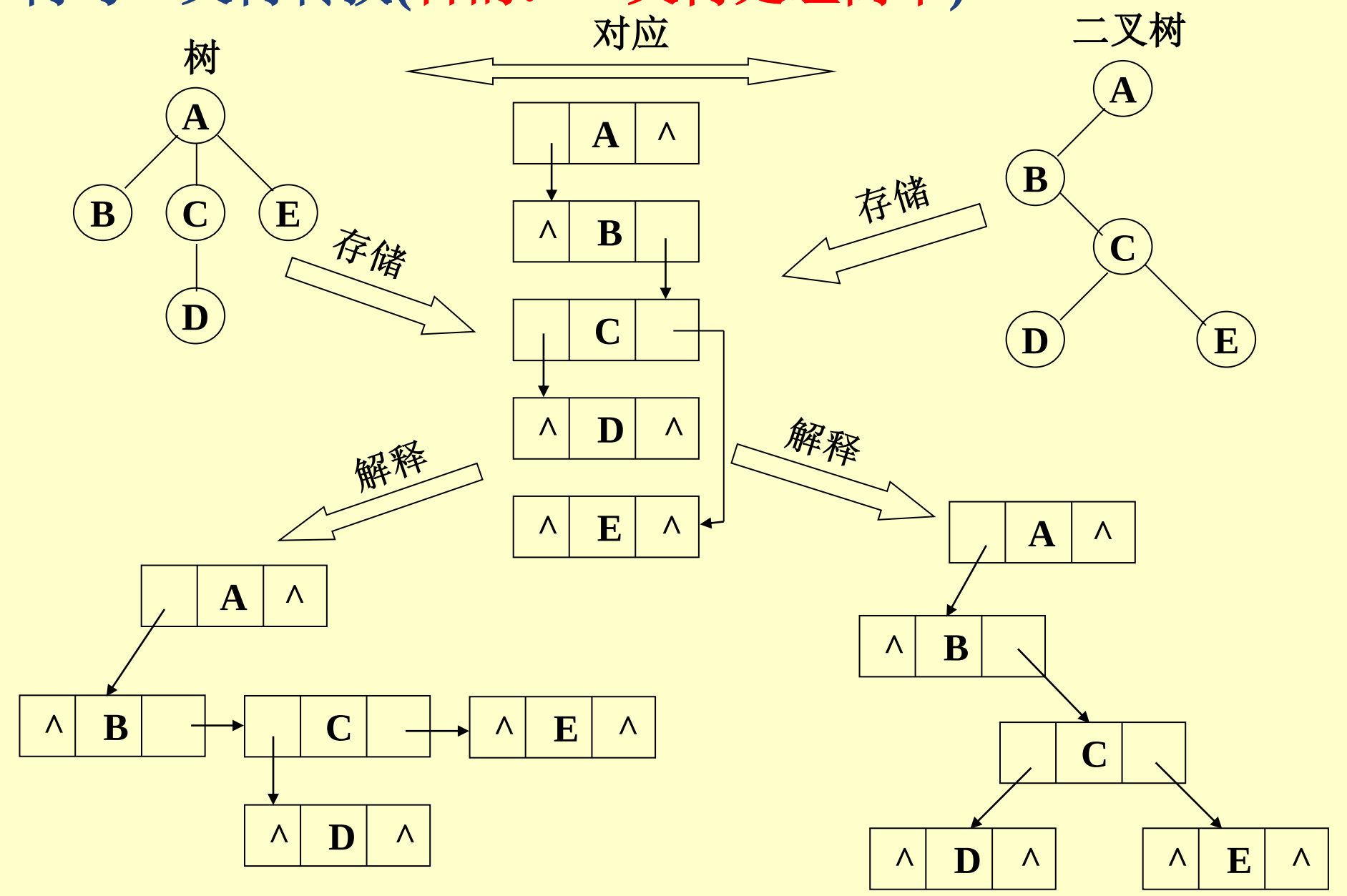
■ 特点

- 操作容易
- 破坏了树的层次

6.4.2 森林与二叉树的转换

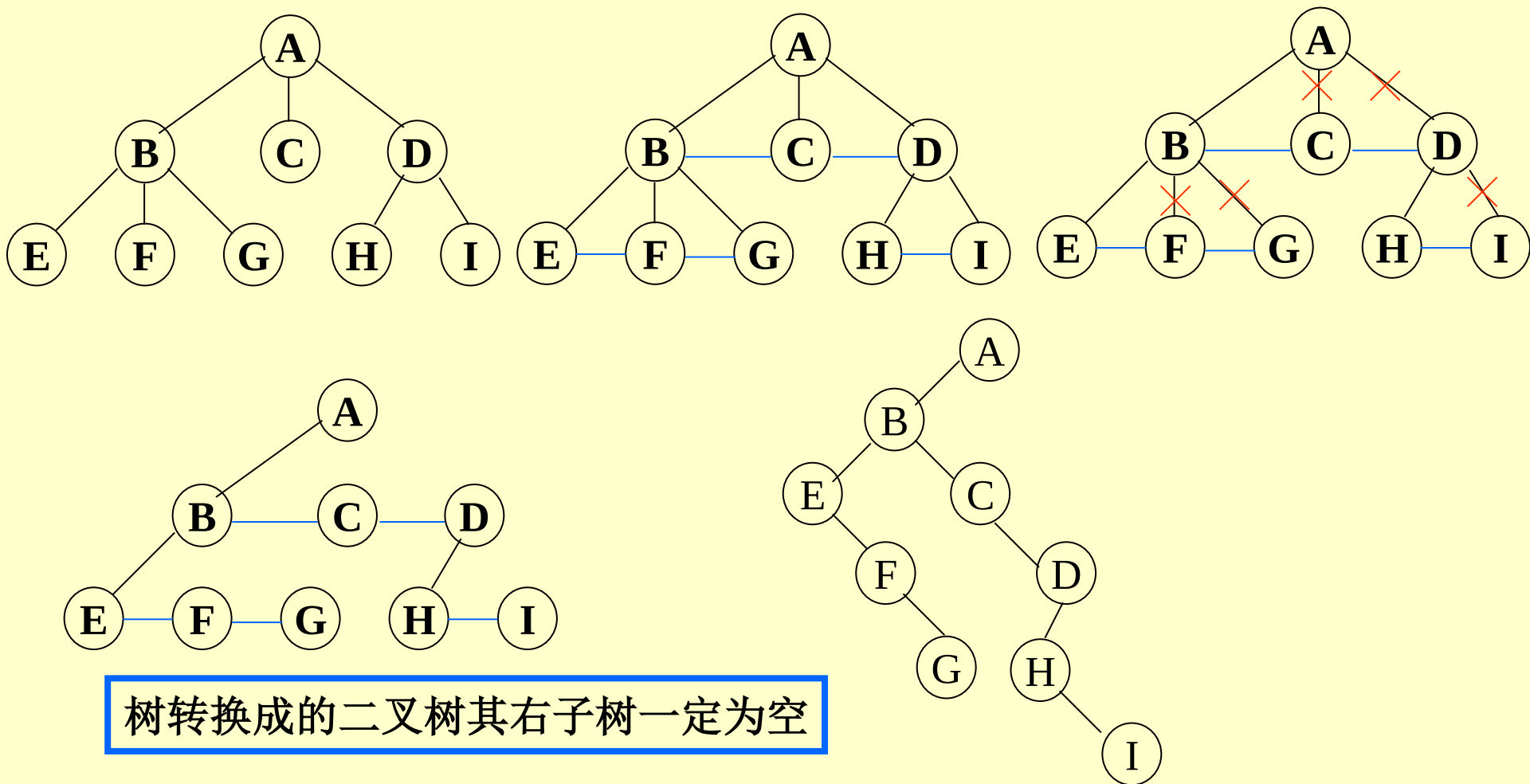
树中一个孩子不分左右

• 树与二叉树转换(目的: 二叉树处理简单)



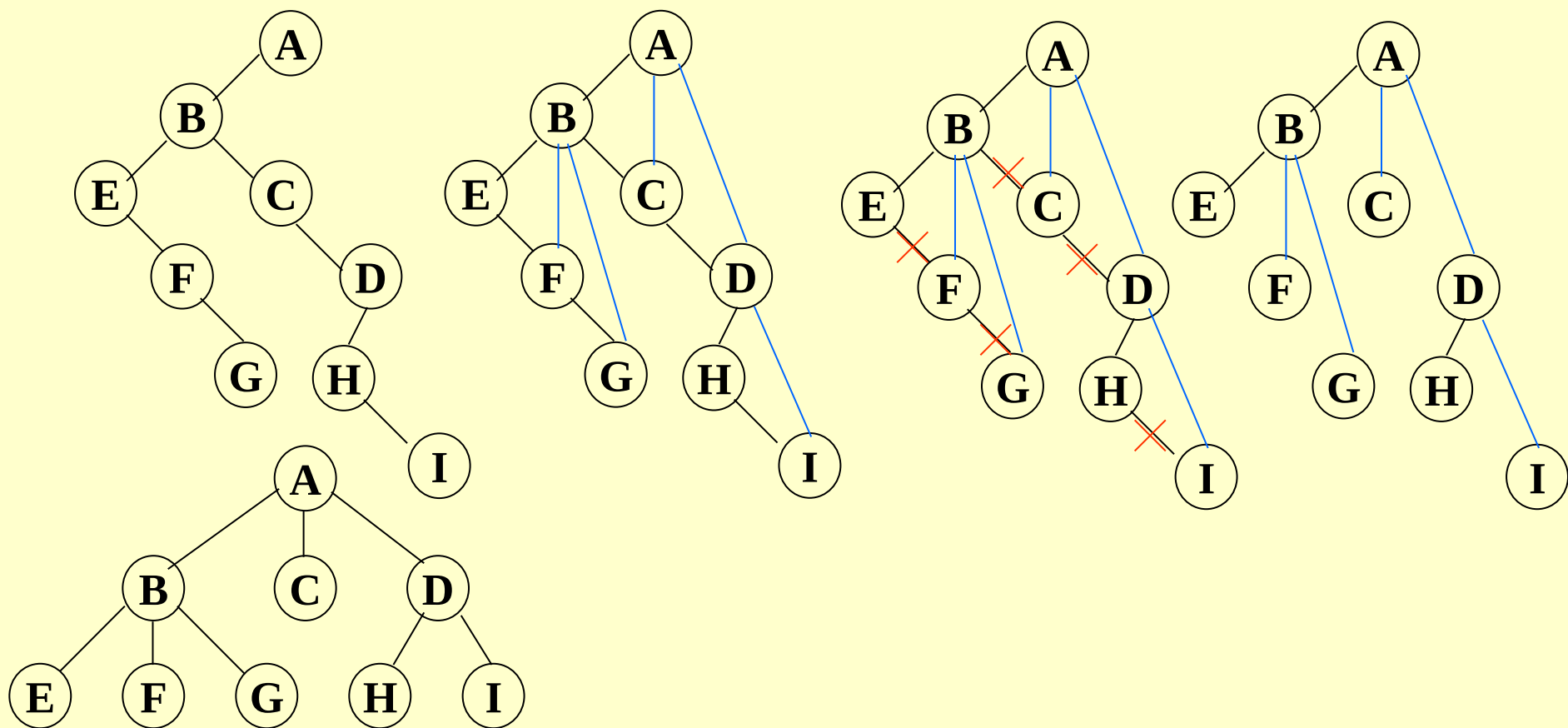
■ 将树转换成二叉树

- 加线：在兄弟之间加一连线
- 抹线：对每个结点，除了其左孩子外，去除其与其余孩子之间的关系
- 旋转：以树的根结点为轴心，将整树顺时针转 45°



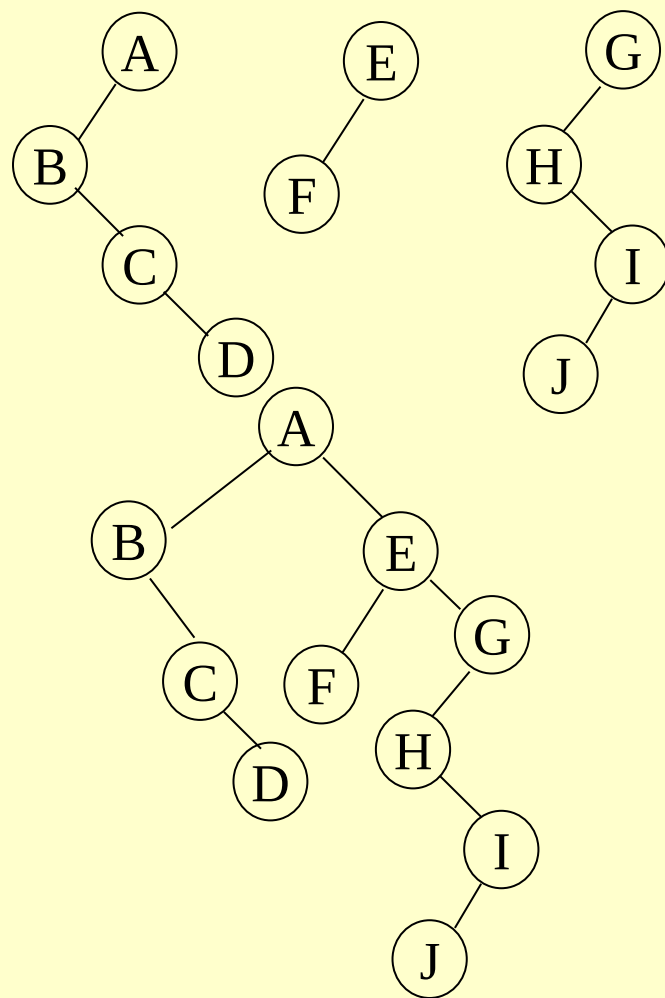
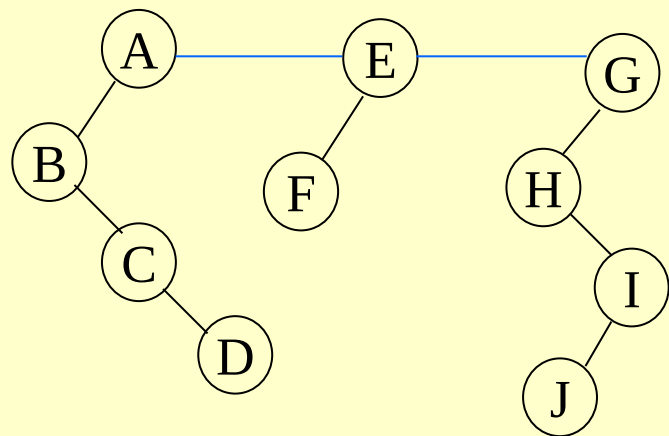
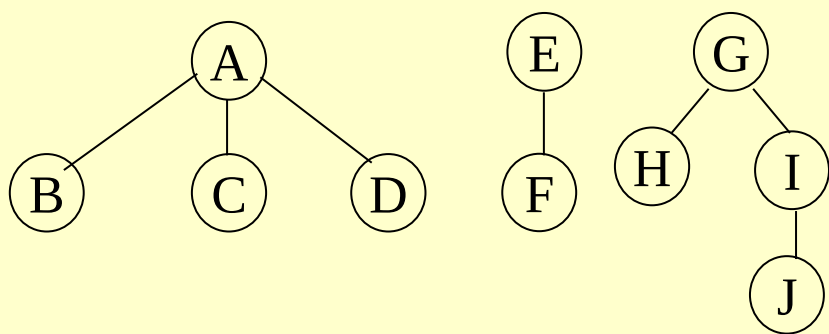
■ 将二叉树转换成树

- 加线：若p结点是双亲结点的左孩子，则将p的右孩子，右孩子的右孩子，.....沿分支找到的所有右孩子，都与p的双亲用线连起来
- 抹线：抹掉原二叉树中双亲与右孩子之间的连线
- 调整：将结点按层次排列，形成树结构



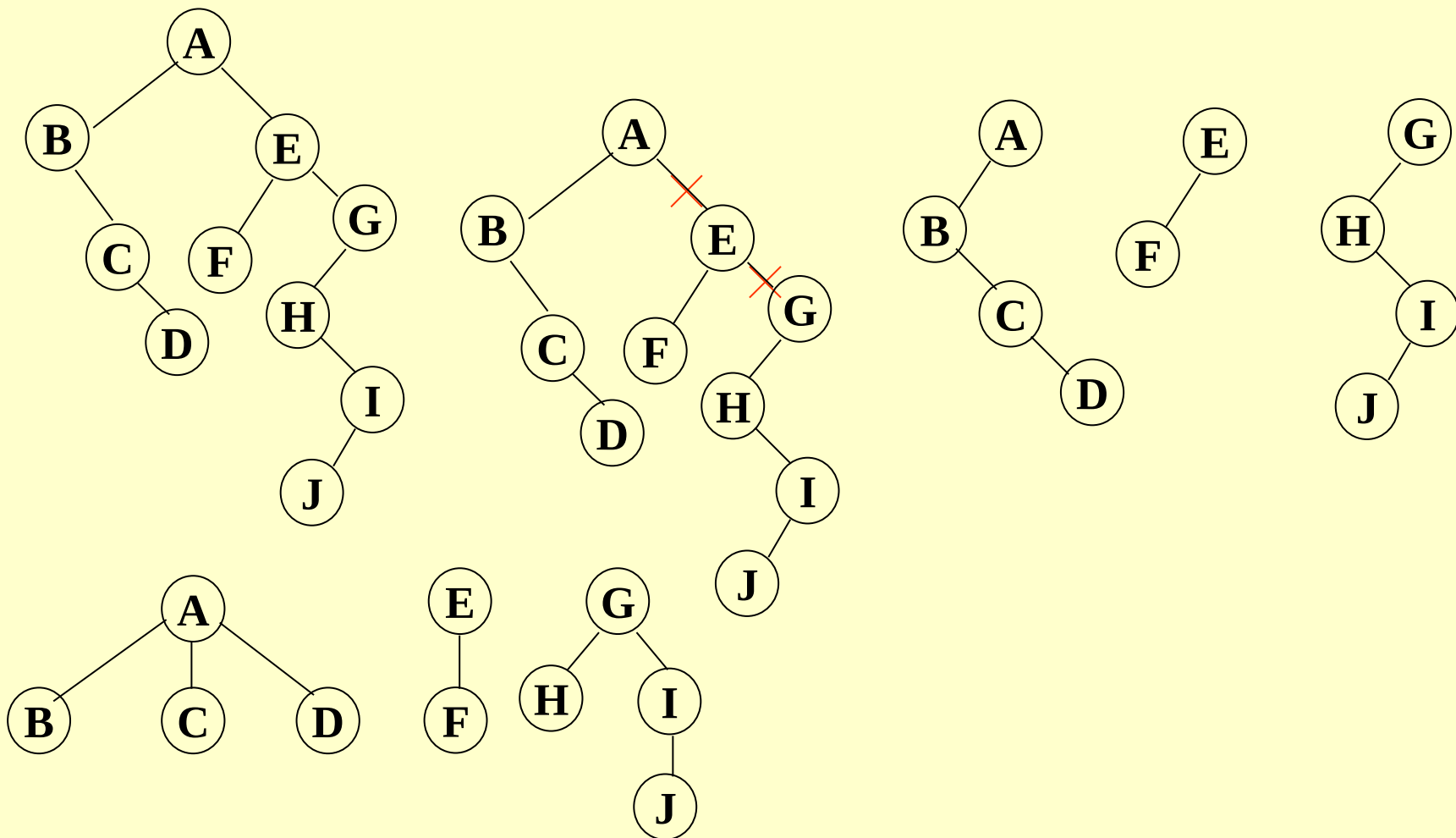
• 森林转换成二叉树

- 将各棵树分别转换成二叉树
- 将每棵树的根结点用线相连
- 以第一棵树根结点为二叉树的根，再以根结点为轴心，顺时针旋转，构成二叉树型结构

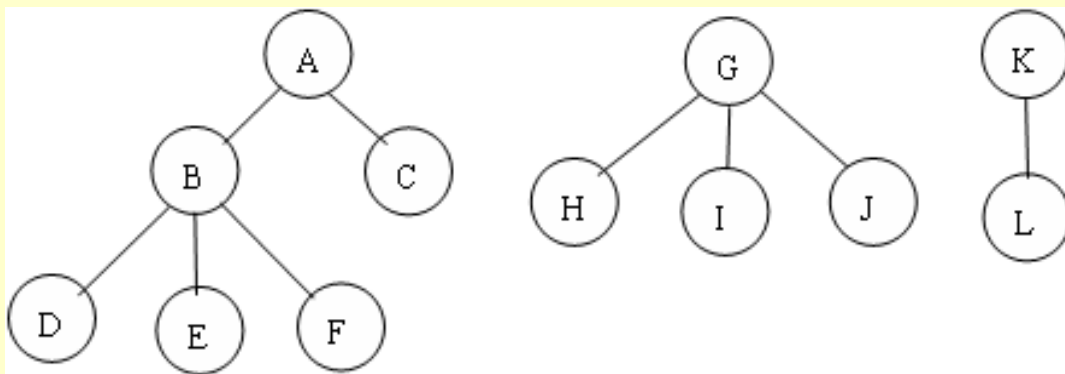


■ 二叉树转换成森林

- 抹线：将二叉树中根结点与其右孩子连线，及沿右分支搜索到的所有右孩子间连线全部抹掉，使之变成孤立的二叉树
- 还原：将孤立的二叉树还原成树



练习1：将图中的森林转化成二叉树



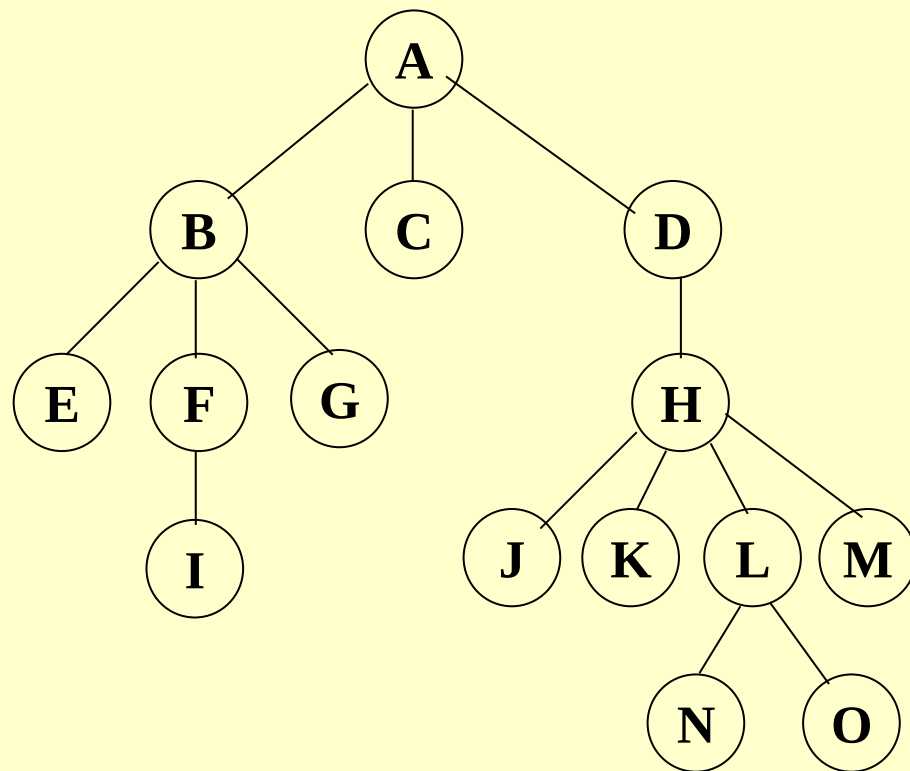
练习2：已知某二叉树的先序序列和中序序列分别为
ABDEHCFIMGJKL和DBHEAIMFCGKLJ，请画这棵二
叉树，并画出二叉树对应的森林。

6.4.3 树和森林的遍历

- 树的遍历

- 常用方法

- 先根（序）遍历：先访问树的根结点，然后依次先根遍历根的每棵子树
 - 后根（序）遍历：先依次后根遍历每棵子树，然后访问根结点
 - 按层次遍历：先访问第一层上的结点，然后依次遍历第二层，……第 n 层的结点



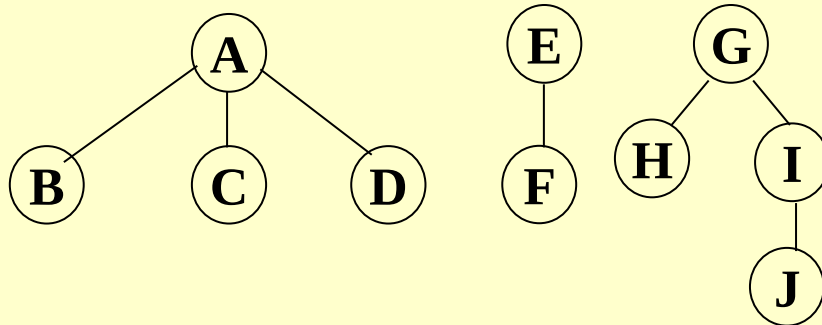
先序遍历: **ABEF I GCDHJ KLNOM**

后序遍历: **EIFGBC JKNOLM HDA**

层次遍历: **ABCDEFGHIJ KLMNO**

2、森林遍历的常用方法

- 先序遍历：先访问第一棵树的根结点，然后依次先序遍历根结点的子树森林，先序遍历剩余子树森林
- 中序遍历：中序遍历第一棵树根结点的子树森林，访问第一棵树的根结点，中序遍历除第一棵树后剩余的子树森林



先序： ABCDEFGHIJ

中序： BCDAFEHJIG

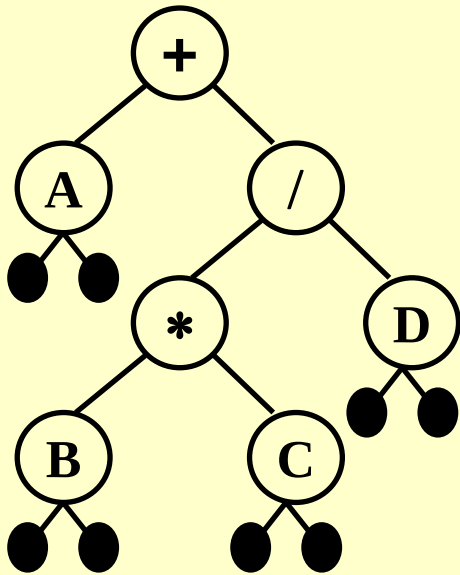
§ 6.6 应用

- **Expression Trees (syntax trees)**
- **Huffman Trees**

❖ 数学表达式

【Example1】 中缀表达式: $a + b * c - d / e$
前缀表达式: $- + a * b c / d e$
后缀表达式: $a b c * + d e / -$

【Example2】 给一个中缀表达式画出二叉树:
 $A + B * C / D$



中序遍历 $\Rightarrow A + B * C / D$

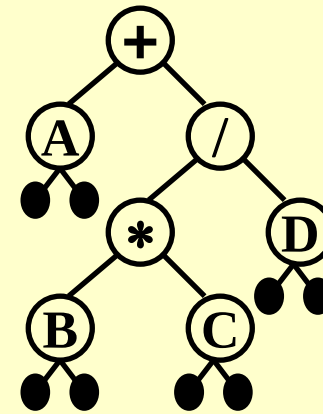
后序遍历 $\Rightarrow A B C * D / +$

前序遍历 $\Rightarrow + A / * B C D$

❖ Expression Trees (syntax trees)

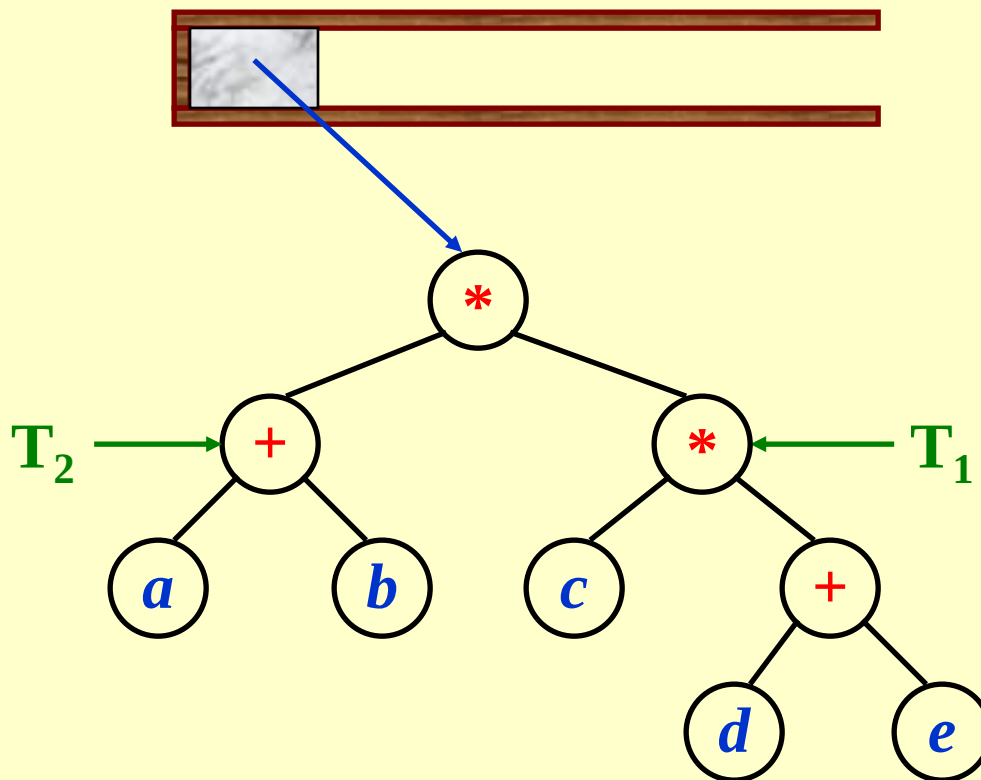
【Example】 给定一个中序表达式:

$$A + B * C / D$$



👉 如何构建一个表达式树
(from postfix expression)

【Example】 $(a + b) * (c * (d + e)) = a b + c d e + * *$



❖ Huffman Tree 哈夫曼树

- A tree with minimum **W**eighted **P**ath **L**ength from the root.

最小化带权路径长度

■ Definitions

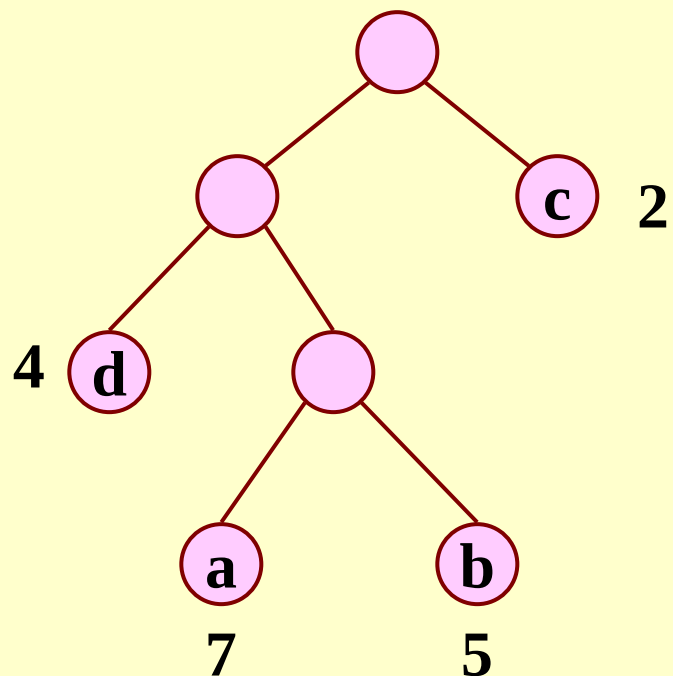
- **路径**: 从树中一个结点到另一个结点之间的分支
- **路径长度**: 路径上的分支数
- **树的路径长度**: 从树根到每一个叶子结点的路径长度之和
- **树的带权路径长度**: 树中所有带权叶子结点的路径长度之和

$$wpl = \sum_{k=1}^n w_k l_k$$

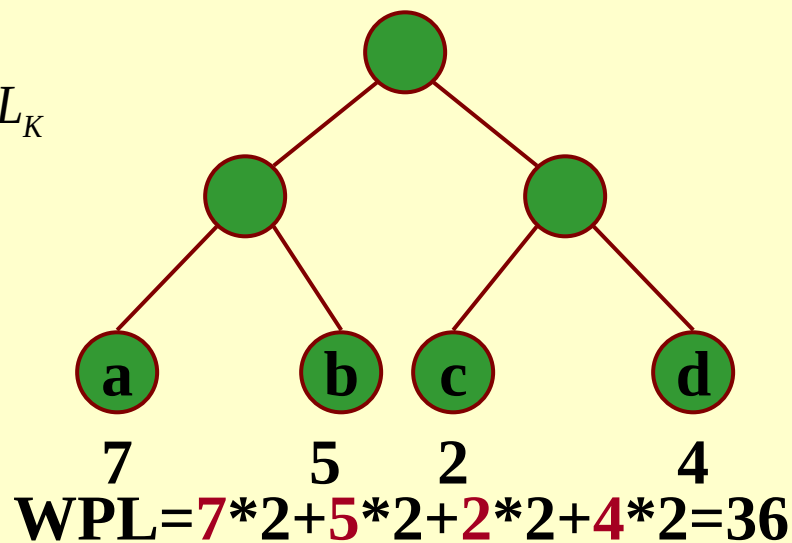
- **Huffman树**——设有n个权值{w1,w2,.....wn}, 构造一棵有n个叶结点的二叉树, 每个叶子的权值为wi, 则wpl最小的二叉树
Huffman树

- 4个结点，权值分别为7，5，2，4，构造有4个叶子结点的二叉树

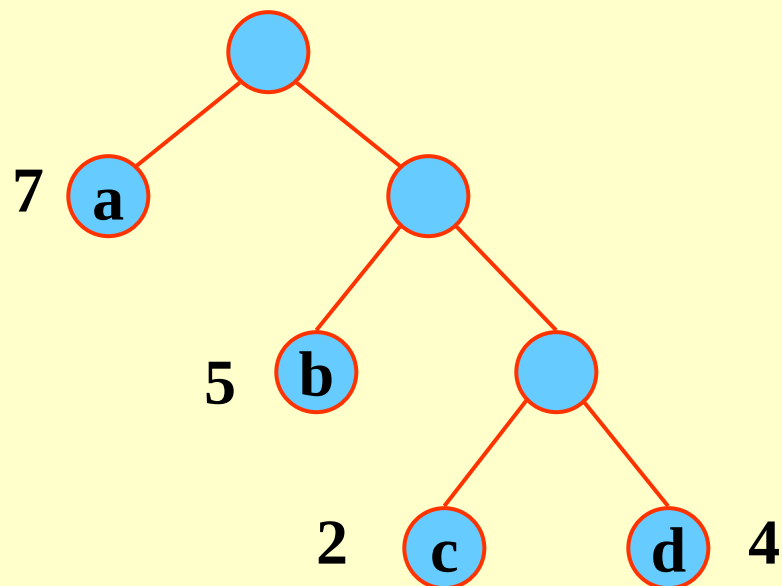
$$WPL = \sum_{k=1}^n W_K L_K$$



$$WPL = 7*3 + 5*3 + 2*1 + 4*2 = 46$$



$$WPL = 7*2 + 5*2 + 2*2 + 4*2 = 36$$



$$WPL = 7*1 + 5*2 + 2*3 + 4*3 = 35$$

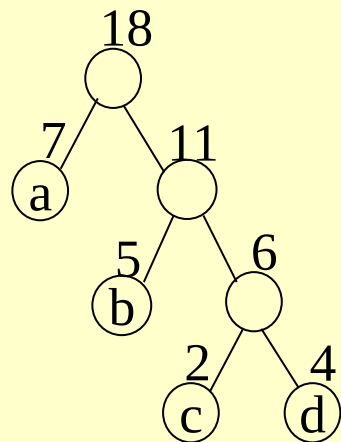
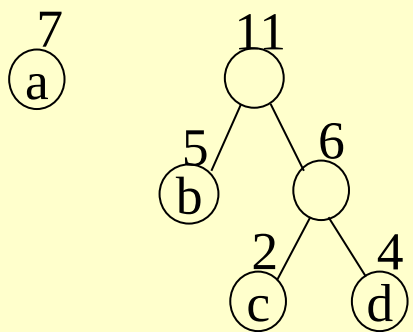
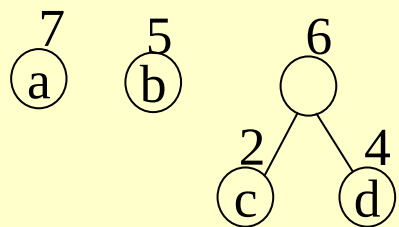
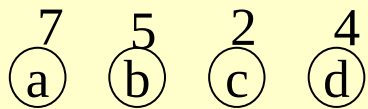
➤ Huffman Tree构建算法

➤ 根据给定的 n 个权值 $\{w_1, w_2, \dots, w_n\}$ ，构造 n 个叶子结点的二叉树

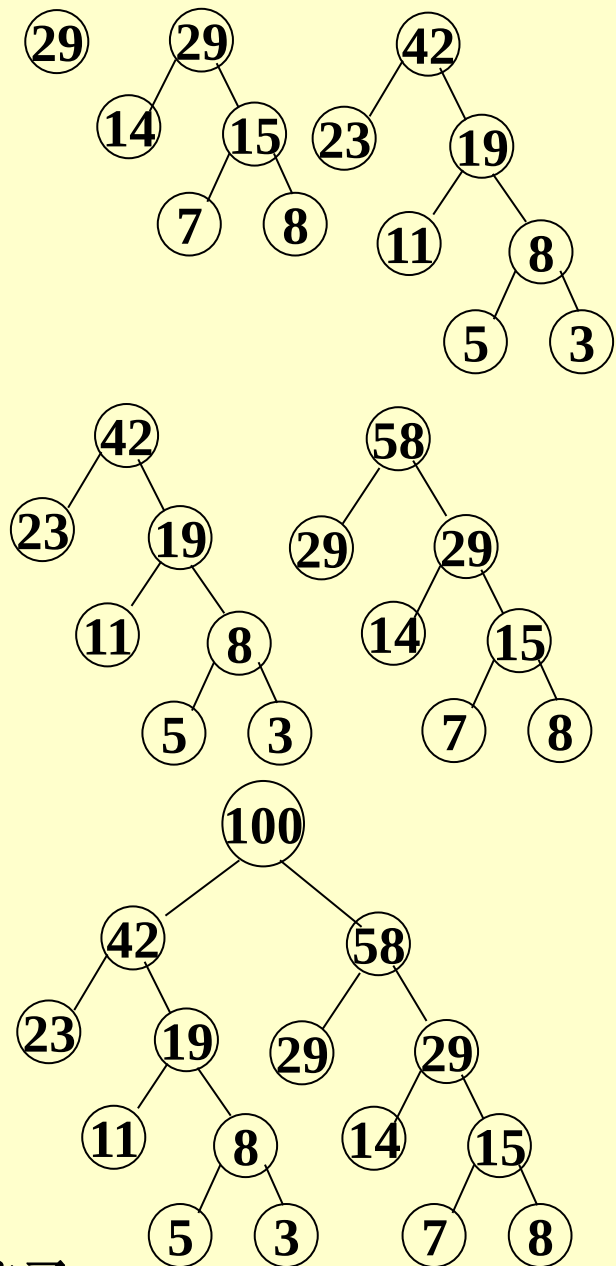
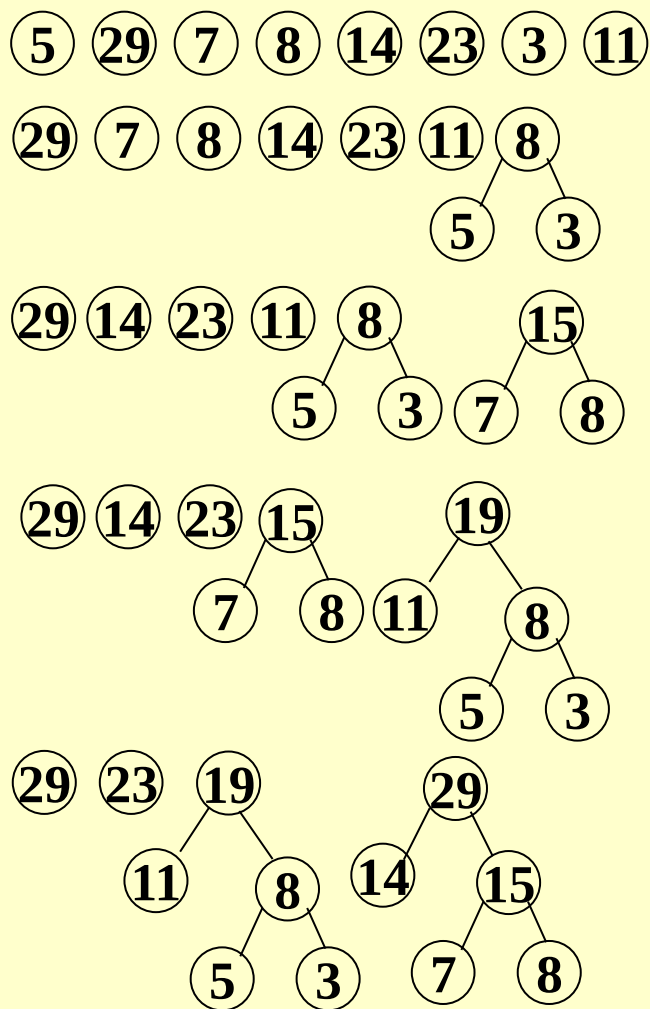
➤ 步骤

- 选择两个具有最小权重的子树合并，生成新的子树，其权重为子树的权重的和。
- 删除两个子树，插入新子树
- 重复上面的步骤直到合并到一个单个树。

Example:



Example $w=\{5, 29, 7, 8, 14, 23, 3, 11\}$

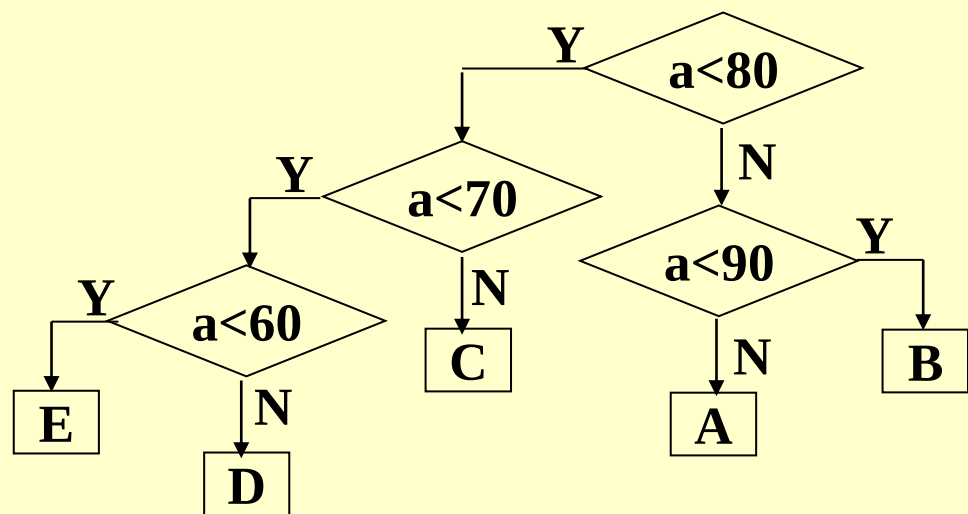
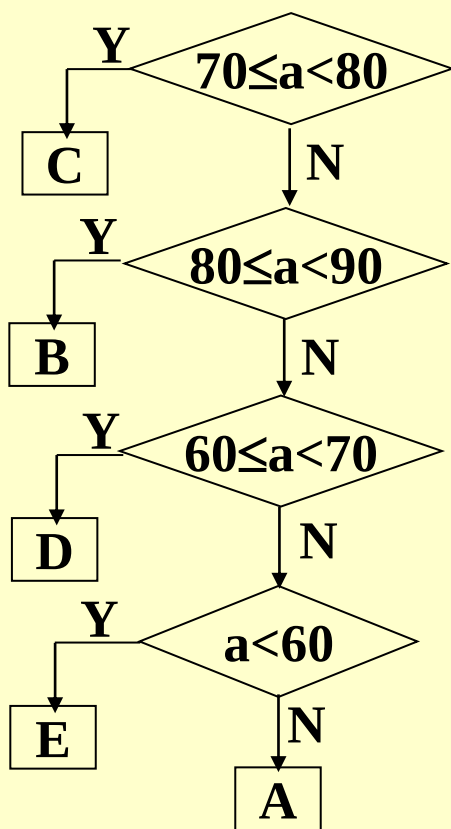
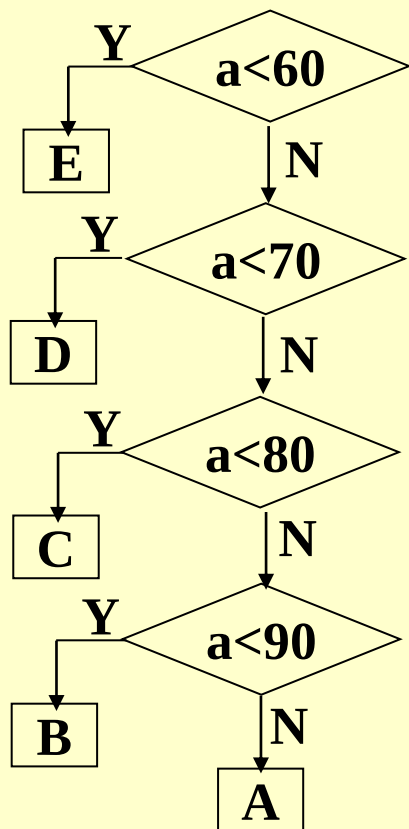
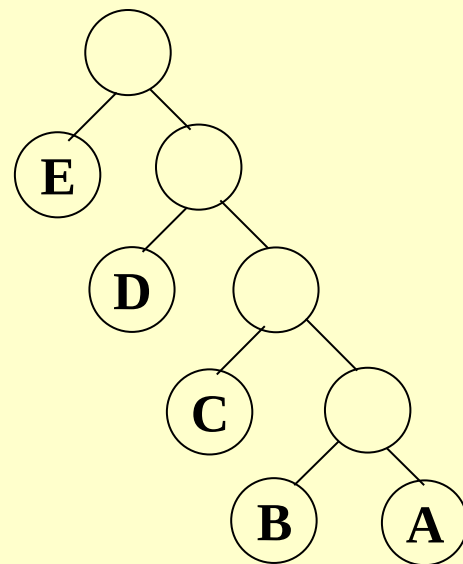


为保持结果唯一性，规则：左孩子小于右孩子。

Huffman Applications应用

□ Best Decision Tree

Grade	E	D	C	B	A
Scores	0~59	60~69	70~79	80~89	90~100
%	0.05	0.15	0.40	0.30	0.10



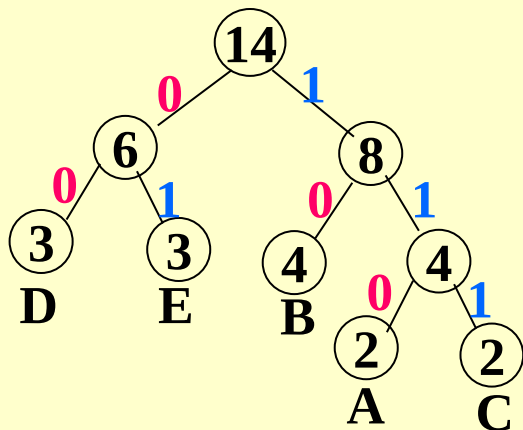
■ Huffman Code: 通信中的编码

给定符号及权重 (概率来衡量).

- 思想: 根据字符出现频率编码, 使电文总长最短
- 编码: 根据字符出现频率构造Huffman树, 然后将树中结点引向其左孩子的分支标“0”, 引向其右孩子的分支标“1”; 每个字符的编码即为从根到每个叶子的路径上得到的0、1序列

$D = \{A, B, C, D, E\}$

$w = \{2, 4, 2, 3, 3\}$



D : 00
E : 01
B : 10
A : 110
C : 111

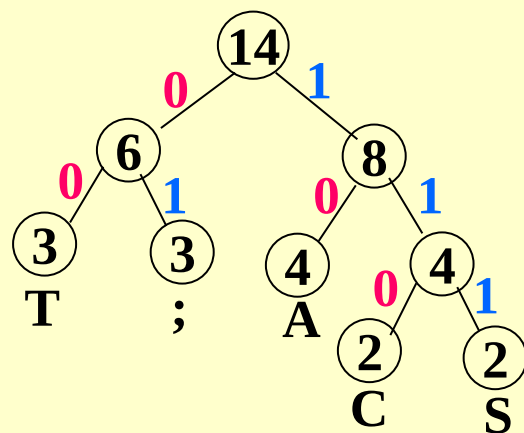
lchild weight rchild parent

1	0	7	0	7
2	0	5	0	6
3	0	2	0	5
4	0	4	0	5
5	3	6	4	6
6	2	11	5	7
k → 7	1	18	6	0

(4)

$x1=1, x2=6 \quad m1=7, m2=11$

■ **Decoding**解码：从Huffman树根开始，从待译码电文中逐位取码。若编码是“0”，则向左走；若编码是“1”，则向右走，一旦到达叶子结点，则译出一个字符；再重新从根出发，直到电文结束



T : 00

; : 01

A : 10

C : 110

S : 111

➤ {CAS;CAT;SAT;AT}

编码：“11010111011101000011111000011000”

➤ Text: “1101000”

解码：“CAT”

练习3: $D=\{A,B,C,D, E \}$, $w=\{4,9,3,6,5\}$, 构建哈夫曼树并给出每个字符的哈夫曼编码。给出电文“110000”的译文。

1. 利用二叉链表存储森林时，根结点右指针是（ ）。
A. 指向最左兄弟 B. 指向最右兄弟 C. 一定为空 D. 不一定为空
2. 设森林F中有3棵树，第一、第二、第三棵树的结点个数分别为M1，M2和M3。与森林F对应的二叉树根结点的右子树上的结点个数是（ ）。
A. M1 B. M1+M2 C. M3 D. M2+M3
3. 设F是一个森林，B是由F变换来的二叉树。若F中有n个非终端结点，则B中右指针域为空的结点有（ ）个。
A. n-1 B. n C. n+1 D. n+2
4. 【2014统考真题】将森林F转换为对应的二叉树T，F中叶结点的个数为（ ）
A. T中叶结点的个数 B. T中度为1的结点个数
C. T中左孩子指针为空的结点个数 D. T中右孩子指针为空的结点个数
5. 某二叉树结点的中序序列为BDAECF，后序序列为DBEFCA，则该二叉树对应的森林包括（ ）棵树。
A. 1 B. 2 C. 3 D. 4
6. 【2016统考真题】若森林F有15条边、25个结点，则F包含树的个数是（ ）
A. 8 B. 9 C. 10 D. 11
7. 【2019统考真题】若将一棵树T转化为对应的二叉树BT，则下列对BT的遍历中，其遍历序列与T的后根遍历序列相同的是（ ）。
A. 先序遍历 B. 中序遍历 C. 后序遍历 D. 按序遍历